

DETAILED DESIGN DOCUMENT

AI-Powered Travel Assistant Planner

Architecture, conversational workflow, state model, and deployment reference for the codebase in `travel_assistant/`.

MSAI-631-B01 — Group 4

Generated from the current codebase (graph.py, state.py, llm.py, app.py)

Companion docs: [design_document.md](#) · [understanding_the_code.md](#)

Contents

Purpose & Scope

System Architecture

- Component diagram

- Why LangGraph instead of an agent loop

- Session isolation

Conversation Workflow (State Machine)

- Full graph diagram

- Preference collection loop

- Confirmation gate

- Generation pipeline

- Post-generation review/edit loop

State Schema

Interaction Sequences

- Single-message happy path

- Multi-turn path with a post-generation edit

Extraction Robustness (small-model hardening)

UI Design

Technology Stack & File Map

Deployment & Runtime Environments

Testing Strategy

Known Limitations / Out of Scope

Design History (Fixed Questionnaire → Adaptive Extraction)

1. Purpose & Scope

The AI-Powered Travel Assistant Planner is a conversational chatbot that collects trip preferences through **natural-language extraction** — not a fixed questionnaire — and generates a day-by-day itinerary, activity recommendations, transportation guidance, a budget estimate, and a packing checklist. It includes a pre-generation confirmation step and a post-generation review/edit loop that regenerates only the affected section(s).

This document describes the system **as implemented** in `travel_assistant/`, covering architecture, the LangGraph conversation flow, the shared state schema, interaction sequences, UI design, technology choices, deployment options, and known limitations.

In scope (per the project proposal): destination selection, adaptive preference collection, itinerary generation, activity recommendations, transportation suggestions, a budget estimate, and a packing checklist — all conversational.

Out of scope: flight/hotel booking, payment processing, visa guidance, live/real-time pricing, and persisting user data or history across sessions.

Document map

File	Role
<code>travel_assistant/state.py</code>	The <code>TravelState</code> TypedDict — the single shared state schema for every node in the graph.
<code>travel_assistant/graph.py</code>	The LangGraph <code>StateGraph</code> — all 15 nodes, extraction logic, generation prompts, and routing.
<code>travel_assistant/llm.py</code>	Wraps the local Hugging Face model as a LangChain chat model; auto-detects CUDA / MPS / CPU.
<code>travel_assistant/app.py</code>	The Gradio <code>ChatInterface</code> — session/thread management and interrupt rendering.
<code>tests/test_graph.py</code>	Control-flow tests against a fake chat model — no GPU or model download required.
<code>notebooks/travel_assistant_colab.ipynb</code>	Runnable copy of the demo for Google Colab (free T4 GPU).

2. System Architecture

2.1 Component diagram

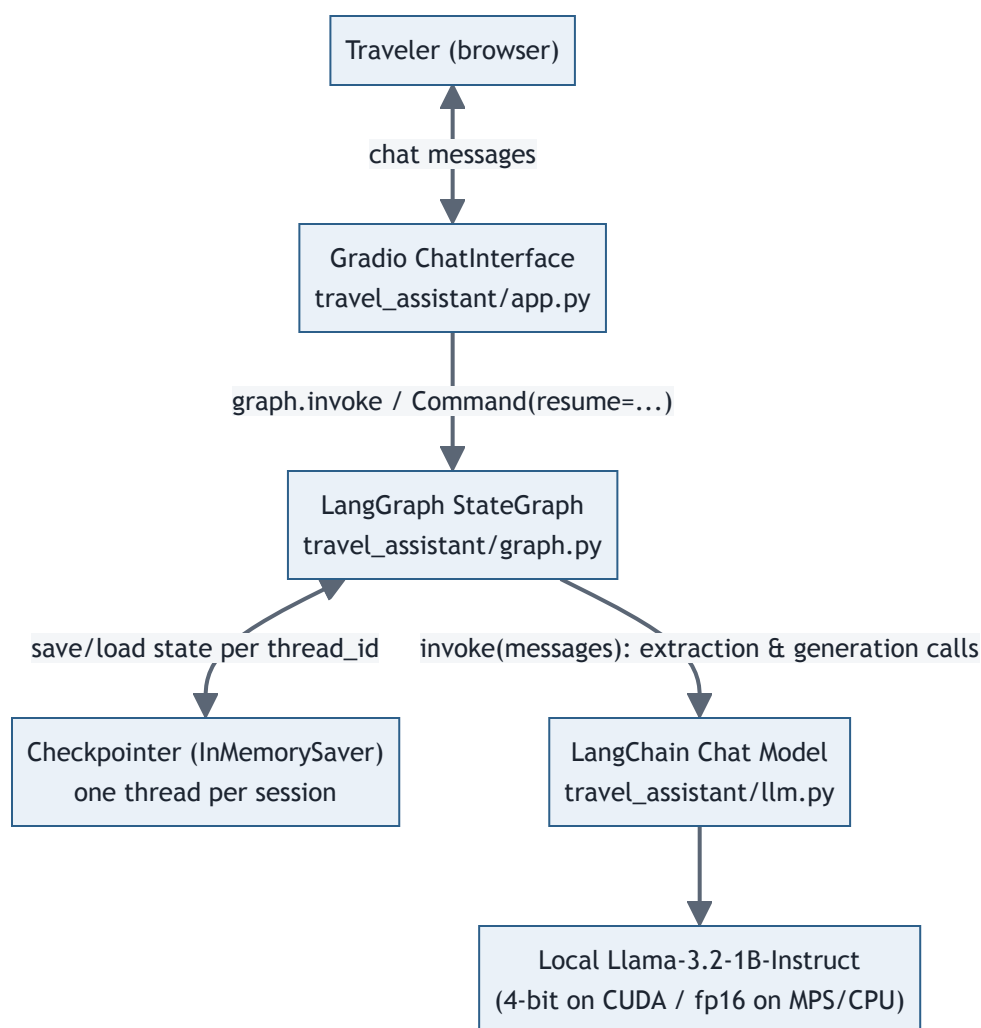


Figure 2.1 — Component-level architecture. The UI never talks to the model directly; every call is mediated by the graph.

2.2 Why LangGraph instead of an agent loop

The chatbot has a clear multi-stage scope: collect preferences → confirm → itinerary → activities → transportation → budget → packing → review. LangGraph models this as an explicit `StateGraph` where **code** controls what happens next via fixed and conditional edges, and the LLM is only ever asked to extract structured fields from one message or generate text for one section at a time.

This is deliberately more constrained than an LLM-driven agent loop, because the project's chosen model (~1B parameters) is not strong enough to reliably plan multi-step tool use on its own. Routing decisions — is a field missing? did the user confirm or ask to edit? — are made by plain Python reading state, never by asking the LLM what to do next.

2.3 Session isolation

Each browser session gets a random `thread_id` (`uuid4` , generated in `app.py`). LangGraph's checkpoint (`InMemorySaver`) keys all state by `thread_id` , so multiple users can chat with the deployed demo concurrently without seeing each other's trips. State does not persist across process restarts — this is intentional per the "no persisting user data across sessions" scope constraint.

3. Conversation Workflow (State Machine)

This is the actual graph compiled by `build_graph()` in `travel_assistant/graph.py` — fifteen nodes, connected by both fixed edges and three conditional-edge decision points.

3.1 Full graph diagram

Shown as two linked diagrams — the preference-collection/confirmation phase, and the generation/review phase it hands off to — so each renders at a readable scale. Together they are the single graph compiled by `build_graph()`; `confirm_preferences` in the first diagram is the same node as the entry point of the second.



Figure 3.1a — Preference collection & confirmation phase (loops until confirmed).

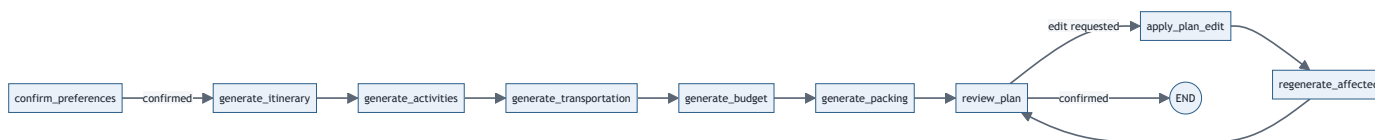


Figure 3.1b — Generation pipeline & post-generation review/edit loop (loops until finalized). Together, Figures 3.1a–b are the complete 15-node graph with 3 conditional-edge decision points: `validate_preferences`, `confirm_preferences`, `review_plan`.

3.2 Preference collection loop

`collect_initial_request` normally just passes through: `app.py` seeds `last_user_input` with the user's very first chat message before the graph starts, so there's real content to extract from immediately. Its `interrupt()` only fires as a fallback (e.g. an empty first message).

`extract_preferences` makes one LLM call per pass: it sends `last_user_input` with a system prompt asking for a JSON object of exactly eight fields (`destination`, `trip_length_days`, `group_type`, `interests`, `budget_level`, `travel_style`, `travel_season`, `must_visit_attractions`), then merges only the non-null, not-yet-known fields into state — a field already set from an earlier turn is never overwritten.

`validate_preferences` is pure Python — *no LLM call*. It recomputes `missing_fields` (the subset of the five required fields still absent) and `collected_fields` (a display-only snapshot of every known field) from scratch every pass, so these derived fields can never drift from the underlying typed state. A conditional edge (`route_after_validation`) reads `missing_fields`: non-empty → `request_missing_fields`; empty → `show_summary`.

`request_missing_fields` interrupts, naming every already-known field and every still-missing required field, and waits for one reply — which can fill in one field or all remaining fields at once. **Graceful degradation:** each extraction pass that finds zero new required fields increments `extraction_attempts`; finding at least one resets it to zero. At 3 unproductive attempts, the node switches to asking about exactly one named field (`kind: "single_field_fallback"`) instead of the full missing list — guarding against an unbounded loop.

3.3 Confirmation gate

`show_summary` is a pass-through that only updates `conversation_stage`. `confirm_preferences` does the actual interrupt, showing the full `collected_fields` snapshot (`kind: "summary_confirmation"`) and waiting for a confirmation or an edit request.

`_is_confirmation()` checks the reply against a fixed set of confirmation words/phrases (matched as whole words, e.g. `"ok"` doesn't false-positive inside `"looking"`), plus a regex pattern for common affirmative sentence *shapes* (e.g. `"all look right," "everything sounds good"`) that aren't literally in the word list. If confirmed, `confirmation_status` → `"confirmed"` and the conditional edge proceeds to `generate_itinerary`. Otherwise the reply is treated as an edit request, routed to `apply_preference_edit` (re-extracts with `overwrite=True`), and loops back to `validate_preferences`.

3.4 Generation pipeline

Once preferences are confirmed, five nodes run automatically in sequence, each making exactly one LLM call:

Node	Reads	Produces
<code>generate_itinerary</code>	destination, trip length, group, interests, budget, optional context	day-by-day itinerary
<code>generate_activities</code>	destination, interests, group, optional context	5–8 bulleted attraction/activity picks
<code>generate_transportation</code>	destination, trip length, group, budget	getting-around guidance
<code>generate_budget</code>	trip length, group, budget level	a deterministic rate-table dollar estimate, narrated by the LLM
<code>generate_packing</code>	destination, trip length, group, interests, optional context	bulleted packing checklist

"Optional context" means any of `travel_style`, `travel_season`, or `must_visit_attractions` the user happened to mention — folded into prompts when present, never required.

Deterministic budget math. The dollar figure comes from a fixed rate table (`_DAILY_RATE_BY_BUDGET` × `_GROUP_MULTIPLIER` × days) in plain Python; the LLM only writes a short narrative around that number. This keeps the one output users are most likely to sanity-check independent of the small model's arithmetic ability.

Each generation node's core logic lives in a plain `*_update(llm, state) -> dict` function (e.g. `_itinerary_update`), separate from the `_make_*_node(llm)` closure that wraps it for the graph. This split exists so `regenerate_affected` (§3.5) can call the same update logic directly on demand, without re-running the whole five-node chain.

3.5 Post-generation review/edit loop

After `generate_packing`, `review_plan` interrupts asking "Would you like any changes, or shall I finalize this plan?" (`kind: "plan_review"`). A confirming reply ends the graph via `route_after_review`. Any other reply routes to `apply_plan_edit`.

`apply_plan_edit` re-runs extraction with `overwrite=True`, using an edit-specific prompt (`_EDIT_EXTRACTION_SYSTEM_PROMPT`) that's given the currently known field values as context and instructed to return each changed field's *full updated value* — merging old and new information rather than replacing it (e.g. `"food, museums"` + `"add outdoor activities"` → `"food, museums, outdoor activities"`).

It then diffs the updated fields against the prior state to find which fields actually changed, and maps each changed field to the generation node(s) it affects via `_FIELD_TO_REGEN_NODES` :

Changed field	Regenerates
<code>destination</code>	itinerary, activities, transportation, packing
<code>trip_length_days</code>	itinerary, budget, packing
<code>group_type</code>	itinerary, transportation, budget
<code>interests</code>	activities, packing
<code>budget_level</code>	budget <i>only</i>
<code>travel_style</code>	itinerary, activities
<code>travel_season</code>	packing
<code>must_visit_attractions</code>	itinerary, activities

The resulting node list (`regeneration_targets`) is deduplicated and reordered to match the canonical generation order. `regenerate_affected` then calls only those nodes' `*_update` functions directly — not the full graph chain — threading each result into the next call's view of state so a request affecting multiple sections sees updates from earlier ones in the same pass. The loop returns to `review_plan` , allowing several rounds of edits before finalizing.

4. State Schema

`travel_assistant/state.py` defines the single `TravelState` shared across every node:

```

class TravelState(TypedDict, total=False):
    # Raw conversational input
    last_user_input: Optional[str]

    # Collected from the traveler – required
    destination: Optional[str]
    trip_length_days: Optional[int]
    group_type: Optional[str]
    interests: Optional[str]
    budget_level: Optional[str]

    # Collected from the traveler – optional
    travel_style: Optional[str]
    travel_season: Optional[str]
    must_visit_attractions: Optional[str]

    # Gap-analysis / conversation bookkeeping
    collected_fields: Dict[str, str]
    missing_fields: List[str]
    conversation_stage: ConversationStage
    confirmation_status: Optional[ConfirmationStatus]
    extraction_attempts: int
    regeneration_targets: Optional[List[str]]

    # Produced by the generation nodes
    itinerary: Optional[str]
    activities: Optional[str]
    transportation: Optional[str]
    budget_estimate: Optional[str]
    packing_list: Optional[str]

```

Field groups

Group	Fields	Notes
Required preferences	<code>destination</code> , <code>trip_length_days</code> , <code>group_type</code> , <code>interests</code> , <code>budget_level</code>	Drive the gap analysis in <code>validate_preferences</code> ; block progress until all 5 are known.
Optional preferences	<code>travel_style</code> , <code>travel_season</code> , <code>must_visit_attractions</code>	Never block progress — folded into prompts if present.
Bookkeeping (derived)	<code>collected_fields</code> , <code>missing_fields</code>	Recomputed from scratch on every <code>validate_preferences</code> pass, never mutated incrementally — cannot drift from the typed fields above.
Bookkeeping (stateful)	<code>conversation_stage</code> , <code>confirmation_status</code> , <code>extraction_attempts</code>	Track where the conversation is and bound the extract/re-ask loop.
Bookkeeping (transient)	<code>regeneration_targets</code>	Populated by <code>apply_plan_edit</code> , consumed and cleared by <code>regenerate_affected</code> .
Generated outputs	<code>itinerary</code> , <code>activities</code> , <code>transportation</code> , <code>budget_estimate</code> , <code>packing_list</code>	Written once by the generation pipeline; selectively overwritten by <code>regenerate_affected</code> .

ConversationStage values

"collecting" → "extracting" → "awaiting_missing_fields" (loop) → "ready" → "awaiting_confirmation" → "reviewing" → "complete". Set by whichever node last ran; purely descriptive, not read by any routing logic.

ConfirmationStatus is "pending" or "confirmed", used at both the pre-generation confirmation gate and the post-generation review gate.

5. Interaction Sequences

Two representative sequences: the common case where a single message supplies every required field, and the case where a follow-up round plus a post-generation edit is needed. Both converge on the same confirmation → generation → review path.

5.1 Single-message happy path

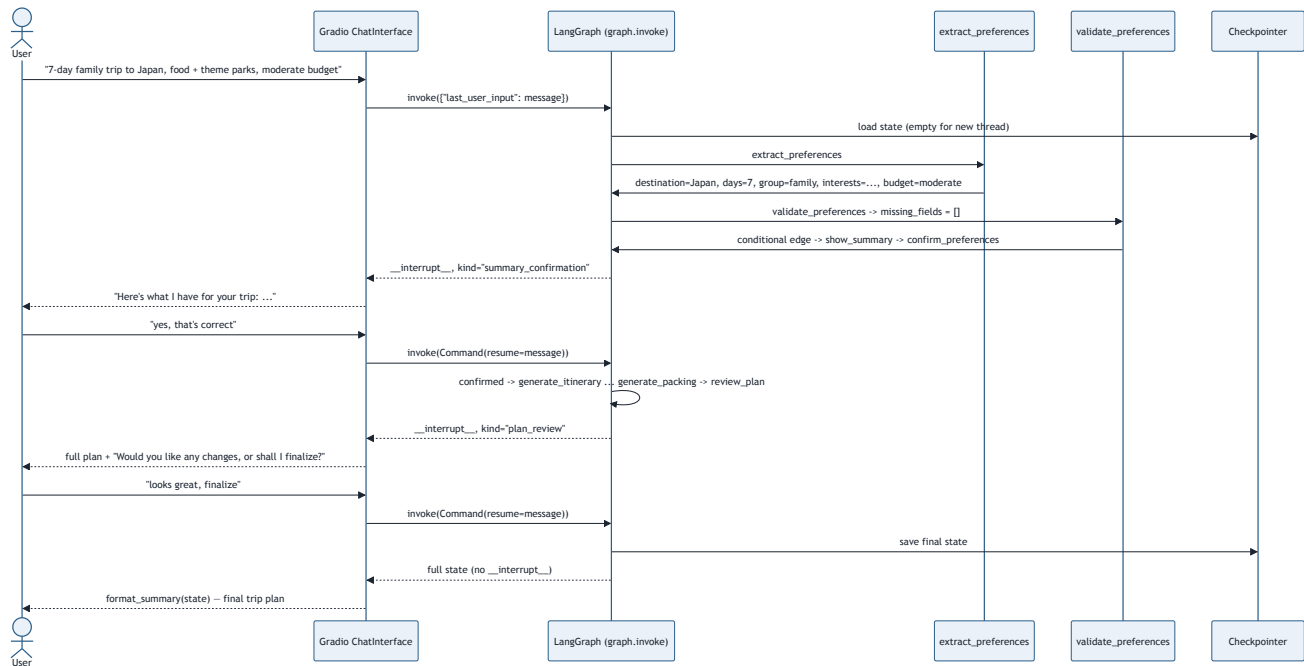


Figure 5.1 — Happy path: three round-trips total (state, confirm, plan-review) when every field is stated up front.

5.2 Multi-turn path with a post-generation edit

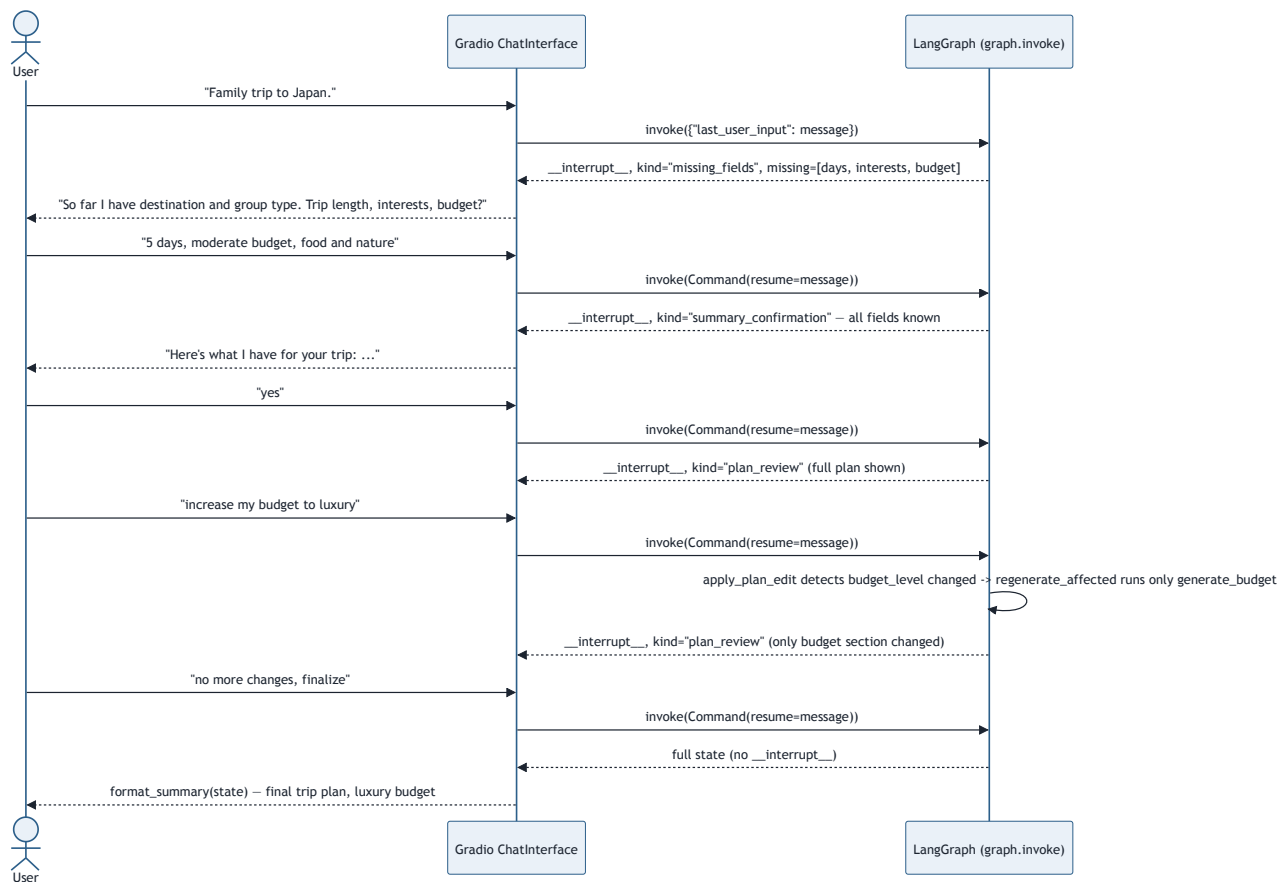


Figure 5.2 — A missing-fields round-trip plus a selective post-generation regeneration (only the budget section is rewritten).

Implementation detail (`travel_assistant/app.py`): `respond()` checks for the `"__interrupt__"` key in the dict returned by `graph.invoke()`. If present, `_render_interrupt()` dispatches on the payload's `"kind"` key (`initial_prompt`, `missing_fields`, `single_field_fallback`, `summary_confirmation`, or `plan_review`) to render the right prompt. If absent, the graph ran to completion and `format_summary()` renders all five generated sections into one message.

6. Extraction Robustness (Small-Model Hardening)

Because the model is only ~1B parameters, `graph.py` contains a substantial layer of code-level backstops around the raw LLM extraction call. These are not incidental — they're the difference between a demo that works reliably and one that silently drifts.

6.1 Truncated JSON repair — `_extract_json`

A small model generating a long field list not infrequently stops before emitting the closing `}`. Rather than requiring a fully balanced object (which would discard every field in a response, not just the one being generated when it cut off), `_extract_json` tracks bracket/string depth character-by-character and repairs a still-open object — closing an unterminated string, dropping a dangling trailing comma, closing every open bracket — before parsing.

6.2 Grounding check — `_is_grounded`

A small model does not reliably follow a "don't guess" instruction. For fields prone to plausible-sounding invention (`group_type` , `budget_level` , `travel_style` , `travel_season`), `_is_grounded` requires that at least one meaningful word from the extracted value actually appears in what the user typed — otherwise the value is discarded rather than trusted. `destination` is deliberately exempt (`_UNGROUNDED_FIELDS`): destinations are often given as nicknames with no word overlap with the formal name (e.g. "the Big Apple" → "New York"), so grounding would reject too many correct extractions there.

6.3 Deterministic extraction decoding

Extraction calls request greedy decoding (`pipeline_kwargs={"do_sample": False}`) rather than the model's normal sampling temperature — sampling could otherwise make the model non-deterministically miss an explicitly stated field on some turns and not others. Generation calls (itinerary, activities, etc.) keep the configured temperature for varied, natural text.

6.4 Numeric and categorical fallbacks

Helper	Handles
<code>_coerce_trip_length</code> / <code>_find_number</code>	Recognizes spelled-out numbers ("seven-day") as a fallback when the model's JSON value is <code>null</code> , not just digit characters.
<code>_infer_group_type</code>	A fixed keyword table (wife/husband/son/daughter/friends/etc.) infers <code>group_type</code> directly from raw text — "my wife and son" → "family" — since the small model doesn't reliably categorize, and the grounding check would reject it even if it did (the word "family" never appears literally).
<code>_find_dollar_amount</code>	If the model paraphrases an explicit dollar figure into a category ("\$4000" → "mid-range") and that category then fails grounding, a literal <code>\$</code> -prefixed amount found in the message is preferred over the invented category.
<code>_looks_like_specific_place</code>	Rejects a <code>must_visit_attractions</code> value that isn't Title Case — real place names extract reliably in Title Case; generic themes misfiled into this field ("food", "hiking") don't — and recovers the value into <code>interests</code> instead.

6.5 Edit-time `group_type` override

During an edit (`overwrite=True`), the model sometimes just echoes back the previous `group_type` unchanged instead of recognizing the edit text as a change (e.g. "will only go byself" left a prior `group_type` of "family" untouched). `_merge_extracted` re-runs `_infer_group_type` on the edit text even when `overwrite=True` , and a keyword match there overrides the model's possibly-stale returned value.

6.6 Extraction attempt bounding

`extraction_attempts` increments on any pass that finds zero new required fields and resets to zero on any pass that finds at least one. At `_MAX_EXTRACTION_ATTEMPTS = 3` , `request_missing_fields` switches from asking for everything still missing to asking about exactly one named field — the graceful-degradation path described in §3.2.

7. UI Design

A single Gradio `ChatInterface` (`travel_assistant/app.py`) — no separate screens or forms. The chat window opens with a pre-seeded assistant greeting introducing what the assistant can do, and invites a free-form trip description rather than a first question.

```
AI-Powered Travel Assistant Planner
Tell me about the trip you would like to plan.

Assistant: Hi! I'm your AI travel planning assistant.
          Tell me about the trip you'd like to take –
          destination, days, who's going, interests,
          and budget – and I'll take care of the rest.
User: 7-day family trip to Japan, we love food and
      theme parks, moderate budget.
Assistant: Here's what I have for your trip: destination
          (Japan), trip length (7 days), group type
          (family), interests (food and theme parks),
          and budget (moderate). Does that all look
          right, or would you like to change anything?
User: yes
Assistant: ## Your trip to Japan
          **Itinerary** ...
          **Activities** ...
          **Getting around** ...
          **Estimated budget** ...
          **Packing checklist** ...
          Would you like any changes, or shall I
          finalize this plan?
User: finalize

[ type a message... ] [ Send ]
```

Figure 7.1 — Chat window mockup, happy path.

If the first message is missing required fields, the assistant responds in plain sentences rather than a form-style list:

```
Great! So far I have your destination (Japan) and group type (family).
Could you tell me your trip length, interests, and budget? Feel free to
share one or all of those in your next message.
```

Both messages are produced by `_render_interrupt()` in `app.py` (`_describe_known` / `_describe_missing` turn field lists into a comma-and-"and" phrase). An earlier revision used a   checklist card for the same information, but that read as a form rather than something a person would say, so it was replaced with conversational prose.

HCI heuristics considered

Heuristic	How it's applied
Visibility of system status	Every response during collection/confirmation names every known and missing field in prose.
Match between system & real world	Opening prompt invites a natural trip description instead of a form-style question.
Recognition rather than recall	Confirmation and review cards show everything already collected.
User control & freedom	Two explicit review points (<code>confirm_preferences</code> , <code>review_plan</code>) let the user correct course without restarting.
Flexibility & efficiency of use	A user who states every field at once resolves in one turn; a partial user is asked only for what's missing.
Error prevention over recovery	<code>_coerce_trip_length</code> and <code>_extract_json</code> degrade gracefully instead of crashing the turn.
Consistency	Every generated section uses the same heading style; the same "known, then missing" phrasing pattern is reused throughout.

8. Technology Stack & File Map

Layer	Choice
Orchestration	LangGraph (<code>StateGraph</code> , <code>interrupt</code> / <code>Command</code> , checkpointer, conditional edges)
Model integration	LangChain (<code>HuggingFacePipeline</code> , <code>ChatHuggingFace</code>)
Model	<code>unsloth/Llama-3.2-1B-Instruct-bnb-4bit</code> (CUDA) or <code>unsloth/Llama-3.2-1B-Instruct</code> fp16 (Apple Silicon MPS / CPU) — auto-detected
UI	Gradio <code>ChatInterface</code>
Runtime environments	Google Colab (free T4 GPU) or a local machine (Apple Silicon / CUDA / CPU)
Version control	GitHub
Testing	<code>pytest</code> , with a fake chat model stub — no GPU or model download required

File map

```
travel_assistant/  
  state.py    <- TravelState TypedDict – the graph's shared state schema  
  llm.py      <- loads the local model as a LangChain chat model  
  graph.py    <- the LangGraph StateGraph and its 15 nodes  
  app.py      <- Gradio ChatInterface wiring  
tests/  
  test_graph.py <- graph control-flow tests using a fake chat model  
notebooks/  
  travel_assistant_colab.ipynb <- runnable Gradio demo for Colab  
docs/  
  design_document.md           <- full architecture write-up (source of this document)  
  understanding_the_code.md    <- plain-English walkthrough, no programming background needed
```

Key function reference

Function	File	Purpose
<code>build_chat_model()</code>	llm.py	Auto-detects CUDA/MPS/CPU and returns a LangChain <code>ChatHuggingFace</code> .
<code>build_graph(llm, checkpointer)</code>	graph.py	Compiles the full 15-node <code>StateGraph</code> .
<code>_merge_extracted(llm, state, overwrite)</code>	graph.py	The single extraction routine used by both initial collection (<code>overwrite=False</code>) and edits (<code>overwrite=True</code>).
<code>_estimate_budget(state)</code>	graph.py	Deterministic rate-table budget calculation, independent of the LLM.
<code>format_summary(state)</code>	graph.py	Renders the five generated sections into one final chat message.
<code>respond(message, history, session)</code>	app.py	Gradio callback — drives <code>graph.invoke</code> / <code>Command(resume=...)</code> per turn.

9. Deployment & Runtime Environments

The system has no server component beyond the Gradio process itself — no database, no external API keys, no persisted storage. Deployment is simply "run the Gradio app somewhere with a Python environment and enough compute to run a 1B-parameter model."

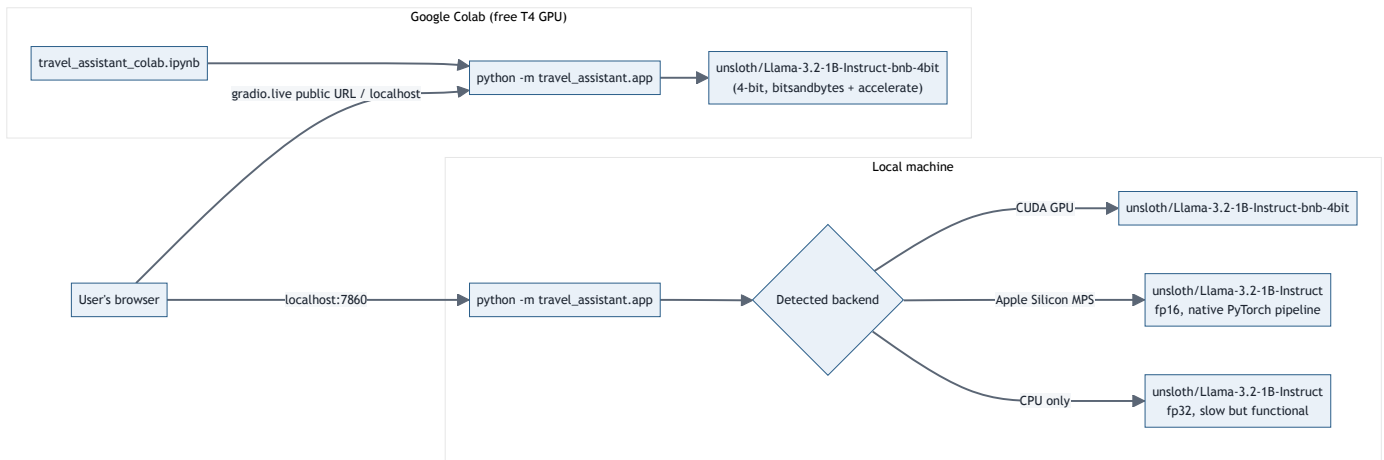


Figure 9.1 — Deployment topology: same codebase, backend auto-selected by `llm.py` at process start.

9.1 Backend auto-detection

`_pick_device()` in `llm.py` checks `torch.cuda.is_available()`, then `torch.backends.mps.is_available()`, falling back to CPU. The model checkpoint choice follows the device:

Backend	Model	Why
CUDA (e.g. Colab T4)	<code>unsloth/Llama-3.2-1B-Instruct-bnb-4bit</code>	The proposal's pre-quantized checkpoint; <code>bitsandbytes</code> 4-bit quantization requires CUDA kernels. Loaded via <code>HuggingFacePipeline.from_model_id(device_map="auto")</code> so <code>accelerate</code> handles dispatch.
Apple Silicon (MPS)	<code>unsloth/Llama-3.2-1B-Instruct (fp16)</code>	<code>bitsandbytes</code> has no Metal backend. Loaded via a pipeline built directly (not <code>from_model_id(device=...)</code> , which only accepts legacy CUDA integer indices, not device strings like <code>"mps"</code>).
CPU	<code>unsloth/Llama-3.2-1B-Instruct (fp32)</code>	Same non-quantized weights; functional but slow. Used automatically when no GPU is present.

9.2 Environments

Google Colab (recommended for teammates without local GPU)

1. Enable the free T4 GPU: Runtime → Change runtime type → T4 GPU.
2. Get the code in — either upload `travel_assistant_project.zip` and unzip, or clone the repo directly from GitHub (always latest `main`).
3. Run all notebook cells top to bottom; the first run downloads the model (a couple of minutes, once per session).
4. Open the printed public `https://xxxxx.gradio.live` link.

Local (Apple Silicon / CUDA / CPU)

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m travel_assistant.app
```

Opens at `localhost:7860`. Verified end-to-end on Apple M4 Max (64 GB) via the MPS backend, no cloud dependency.

Logic-only (no model download, for CI / quick checks)

```
pip install -r requirements.txt
pytest tests/
```

Keeping the zip snapshot fresh. `travel_assistant_project.zip` is a point-in-time snapshot for Colab Option A, not a live link to the code. If `travel_assistant/` changes, regenerate it before teammates rely on it — command is in the README.

10. Testing Strategy

`tests/test_graph.py` is the only automated check in the repo — there is no linter or formatter configured. Tests never download the real model or need a GPU: a fake chat model branches on the system prompt text — extraction calls get a small regex-based fake extractor, generation calls get an echoed prompt snippet.

```
pytest tests/
pytest tests/test_graph.py::test_post_generation_edit_regenerates_only_the_affected_section
```

What this validates: the right fields get extracted per turn; nothing gets asked for twice; editing one field doesn't erase another; the missing-fields/confirmation/review interrupts fire with the correct `kind` and payload; and — the most safety-critical behavior — that requesting a change to one part of the finished plan (e.g. "increase my budget") regenerates *only* the mapped section(s) via `_FIELD_TO_REGEN_NODES`, leaving the rest of the plan's text byte-for-byte untouched.

What tests do *not* cover: actual generation quality/factual accuracy of the local model's output, or performance under real GPU load — those require the full demo (Colab or local) and manual review, not automated assertions.

11. Known Limitations / Out of Scope

Per the proposal: no flight/hotel booking, no payment processing, no visa guidance, no live pricing or availability, not a replacement for a travel agent. Additionally, as currently designed:

- **Extraction quality ceiling.** Depends on the local 1B model's ability to follow the JSON-output instruction reliably. `extraction_attempts` bounds the worst case (falls back to a single named field after 3 unproductive attempts) but doesn't eliminate occasional re-prompts if the model's JSON is malformed.
- **interests vs. must_visit_attractions confusion.** The model frequently misfiles a generic theme ("food") into `must_visit_attractions` instead of `interests`. `_looks_like_specific_place` (\$6.4) recovers most of these, but the underlying confusion is a model limitation, not something fully eliminated.

- **Confirmation-phrase coverage.** `_is_confirmation()` matches a fixed word/phrase list plus one regex pattern; a phrasing outside both is treated as an edit request instead — which, if it contains no recognizable field change, leaves state unchanged and can look like the assistant ignoring a confirmation.
- **No preference value validation.** Beyond `trip_length_days`, answers are taken at face value — there's no check that `budget_level` is one of budget/moderate/luxury; an out-of-vocabulary value defaults to the "moderate" rate in `_estimate_budget`.
- **No mid-collection jump-back.** There's no way to jump back into mid-collection editing before the summary is reached short of continuing to answer — but the summary and post-generation review stages both fully support corrections.
- **Model output quality.** Weaker factual grounding than a larger model — acceptable for this course project's scope, worth noting in a results report.
- **No session persistence.** State lives only in the in-memory checkpointer for the life of the process; restarting the app loses all in-progress conversations (by design — out of scope per the proposal).

12. Design History: Fixed Questionnaire → Adaptive Extraction

The Week 2 implementation used a **fixed sequential questionnaire**: one field asked at a time, in hard-coded order, regardless of what the user had already volunteered. It worked, but read as a form rather than a conversational assistant.

12.1 The redesign argument

Replace question-by-question delivery with **adaptive information extraction**: the user describes their trip in free text; one LLM call extracts whatever structured fields it can; a pure-Python gap analysis determines what's still missing; the system asks only for that, accepting any number of fields in a single reply, in any order.

Heuristic	Fixed questionnaire	Adaptive extraction
Visibility of system status	One question at a time; no overall progress sense	Every turn shows all required fields, known and missing
Match between system & real world	Plain-language but rigid one-field-per-turn	Mirrors describing a trip to a human agent in one message
User control & freedom	Cannot answer out of order	Any subset of fields, any order, across 1+ messages
Flexibility & efficiency	A front-loading user gets no benefit — still asked all 5	Front-loading skips the clarification loop entirely
Recognition rather than recall	Must recall what's already been asked	Response always restates what's already known

Interaction cost drops from a fixed $O(5)$ turns to $O(\text{missing fields after the first message})$ — typically 0–2 follow-up turns for a message that states most fields up front.

12.2 What was, and wasn't, built from the earlier docx proposal

Proposed	Status
Post-generation review/modify with selective regeneration	Built — as <code>review_plan</code> → <code>apply_plan_edit</code> → <code>regenerate_affected</code> , same idea, different node names.
Input-validation-by-degradation (invalid response → re-ask)	Built — via <code>extraction_attempts</code> bounding, not a dedicated validation node.
<code>must_visit_attractions</code> optional field	Built exactly as proposed (with the recovery logic in §6.4 needed in practice).
Pre-generation confirmation gate	Built — added during implementation, one stage earlier than the docx's own review stage.
<code>user_name</code> / name personalization	Not built — no such field in <code>TravelState</code> .
<code>recommend_destination</code> branch	Not built — <code>destination</code> is a required field like any other.
Separate <code>vacation_type</code> field	Deliberately rejected — duplicate of <code>interests</code> .
Separate "destination known vs. undecided" boolean	Deliberately rejected — redundant with <code>missing_fields</code> already expressing <code>destination is None</code> .